

Part A — Regression Model

```

1  # =====
2  # PART A: HOUSE PRICE PREDICTION USING LINEAR REGRESSION
3  # =====
4
5  import pandas as pd
6  import numpy as np
7  import matplotlib.pyplot as plt
8
9  from sklearn.model_selection import train_test_split
10 from sklearn.linear_model import LinearRegression
11 from sklearn.metrics import mean_absolute_error
12 from sklearn.metrics import mean_squared_error
13 from sklearn.metrics import r2_score
14
15
16 # 1. Create dataset
17 data = pd.DataFrame({
18     "area": [
19         800, 900, 1000, 1100, 1200,
20         1300, 1400, 1500, 1600, 1700,
21         1800, 1900, 2000, 2200, 2400,
22         2600, 2800, 3000, 3200, 3500
23     ],
24
25     "bedrooms": [
26         1, 2, 2, 2, 2,
27         3, 3, 3, 3, 3,
28         3, 4, 4, 4, 4,
29         4, 5, 5, 5, 6
30     ],
31
32     "bathrooms": [
33         1, 1, 1, 2, 2,
34         2, 2, 2, 2, 2,
35         3, 3, 3, 3, 3,
36         3, 4, 4, 5, 5
37     ],
38
39     "price": [
40         40, 45, 50, 58, 65,
41         72, 78, 85, 92, 100,
42         108, 118, 128, 142, 155,
43         170, 185, 205, 225, 250
44     ]
45 })
46
47
48 print("HOUSE PRICE DATASET")
49 print(data)
50
51
52 # 2. Select features and target
53 X = data[["area", "bedrooms", "bathrooms"]]
54 y = data["price"]
55
56
57 # 3. Train-test split
58 X_train, X_test, y_train, y_test = train_test_split(
59     X,
60     y,
61     test_size=0.2,
62     random_state=42
63 )
64
65
66 # 4. Create model
67 model = LinearRegression()
68
69
70 # 5. Train model
71 model.fit(X_train, y_train)
72
73

```

```
73
74 # 6. Prediction
75 y_pred = model.predict(X_test)
76
77
78 # 7. Evaluation
79 mae = mean_absolute_error(y_test, y_pred)
80 mse = mean_squared_error(y_test, y_pred)
81 rmse = np.sqrt(mse)
82 r2 = r2_score(y_test, y_pred)
83
84
85 print("\n===== MODEL PERFORMANCE =====")
86
87 print("MAE :", round(mae, 2))
88 print("MSE :", round(mse, 2))
89 print("RMSE :", round(rmse, 2))
90 print("R2 :", round(r2, 4))
91
92
93 # 8. Actual vs Predicted
94 results = pd.DataFrame({
95     "Actual Price": y_test.values,
96     "Predicted Price": np.round(y_pred, 2)
97 })
98
99 print("\n===== ACTUAL VS PREDICTED =====")
100 print(results)
101
102
103 # 9. Graph
104 plt.figure(figsize=(7, 5))
105
106 plt.scatter(y_test, y_pred)
107
108 plt.xlabel("Actual House Price")
109 plt.ylabel("Predicted House Price")
110
111 plt.title("Actual vs Predicted House Prices")
112
113 plt.show()
```

HOUSE PRICE DATASET

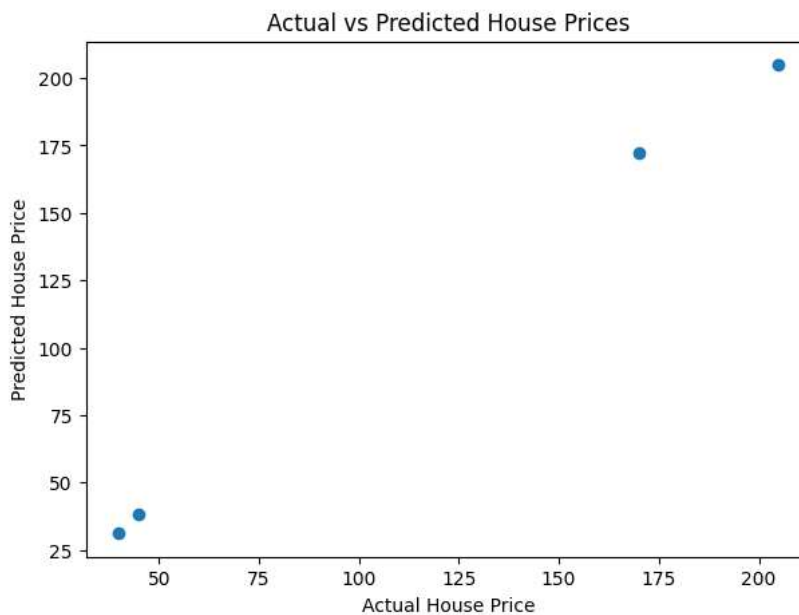
	area	bedrooms	bathrooms	price
0	800	1	1	40
1	900	2	1	45
2	1000	2	1	50
3	1100	2	2	58
4	1200	2	2	65
5	1300	3	2	72
6	1400	3	2	78
7	1500	3	2	85
8	1600	3	2	92
9	1700	3	2	100
10	1800	3	3	108
11	1900	4	3	118
12	2000	4	3	128
13	2200	4	3	142
14	2400	4	3	155
15	2600	4	3	170
16	2800	5	4	185
17	3000	5	4	205
18	3200	5	5	225
19	3500	6	5	250

===== MODEL PERFORMANCE =====

MAE : 4.48
MSE : 32.52
RMSE : 5.7
R2 : 0.994

===== ACTUAL VS PREDICTED =====

	Actual Price	Predicted Price
0	40	31.20
1	205	204.90
2	170	172.03
3	45	38.03

**Part B — Gradient Descent**

```

1 # =====
2 # PART B: GRADIENT DESCENT FOR LINEAR REGRESSION
3 # =====
4
5 import numpy as np
6 import pandas as pd
7 import matplotlib.pyplot as plt
8
9
10 # 1. House area and price
11 X = np.array([
12     8, 9, 10, 11, 12,
13     13, 14, 15, 16, 17,
14     18, 19, 20, 22, 24,
15     26, 28, 30, 32, 35

```

```

16 ], dtype=float)
17
18 y = np.array([
19     40, 45, 50, 58, 65,
20     72, 78, 85, 92, 100,
21     108, 118, 128, 142, 155,
22     170, 185, 205, 225, 250
23 ], dtype=float)
24
25
26 # 2. Standardize input
27 X_mean = np.mean(X)
28 X_std = np.std(X)
29
30 X_scaled = (X - X_mean) / X_std
31
32
33 # 3. Initialize parameters
34 w = 0.0
35 b = 0.0
36
37
38 # 4. Hyperparameters
39 learning_rate = 0.01
40 iterations = 1000
41
42
43 # 5. Store cost
44 cost_history = []
45
46 n = len(X_scaled)
47
48
49 # 6. Gradient Descent
50 for i in range(iterations):
51
52     # Prediction
53     y_pred = w * X_scaled + b
54
55     # Error
56     error = y_pred - y
57
58     # Cost
59     cost = np.mean(error ** 2)
60
61     cost_history.append(cost)
62
63     # Gradients
64     dw = (2 / n) * np.sum(X_scaled * error)
65
66     db = (2 / n) * np.sum(error)
67
68     # Update parameters
69     w = w - learning_rate * dw
70
71     b = b - learning_rate * db
72
73
74 # 7. Final prediction
75 y_pred = w * X_scaled + b
76
77
78 # 8. Performance metrics
79 mae = np.mean(np.abs(y - y_pred))
80
81 mse = np.mean((y - y_pred) ** 2)
82
83 rmse = np.sqrt(mse)
84
85 ss_total = np.sum((y - np.mean(y)) ** 2)
86
87 ss_residual = np.sum((y - y_pred) ** 2)
88
89 r2 = 1 - (ss_residual / ss_total)
90
91
92 # 9. Display results

```

```
93 print("==== GRADIENT DESCENT RESULTS ====")
94
95 print("Learning Rate :", learning_rate)
96 print("Iterations      :", iterations)
97
98 print("Final Weight  :", round(w, 4))
99 print("Final Bias    :", round(b, 4))
100
101 print("Final Cost     :", round(cost_history[-1], 4))
102
103 print("\n==== PERFORMANCE METRICS ====")
104
105 print("MAE   :", round(mae, 4))
106 print("MSE   :", round(mse, 4))
107 print("RMSE  :", round(rmse, 4))
108 print("R2    :", round(r2, 4))
109
110
111 # 10. Actual vs Predicted
112 results = pd.DataFrame({
113     "Area": X,
114     "Actual Price": y,
115     "Predicted Price": np.round(y_pred, 2)
116 })
117
118 print("\n==== ACTUAL VS PREDICTED ====")
119
120 print(results)
121
122
123 # 11. Cost vs Iterations
124 plt.figure(figsize=(8, 5))
125
126 plt.plot(cost_history)
127
128 plt.xlabel("Iterations")
129
130 plt.ylabel("Cost")
131
132 plt.title("Gradient Descent - Cost vs Iterations")
133
134 plt.show()
135
136
137 # 12. Regression graph
138 plt.figure(figsize=(8, 5))
139
140 plt.scatter(X, y, label="Actual Data")
141
142 plt.plot(
143     X,
144     y_pred,
145     label="Gradient Descent Prediction"
146 )
147
148 plt.xlabel("House Area (100 sq.ft)")
149
150 plt.ylabel("House Price (Lakhs)")
151
152 plt.title("Linear Regression using Gradient Descent")
153
154 plt.legend()
155
```

===== GRADIENT DESCENT RESULTS =====

Learning Rate : 0.01
Iterations : 1000
Final Weight : 60.7498
Final Bias : 118.55
Final Cost : 10.7116

===== PERFORMANCE METRICS =====

MAE : 2.6697
MSE : 10.7116
RMSE : 3.2729
R2 : 0.9971

===== ACTUAL VS PREDICTED =====

	Area	Actual Price	Predicted Price
0	8.0	40.0	33.27
1	9.0	45.0	41.06
2	10.0	50.0	48.85
3	11.0	58.0	56.64
4	12.0	65.0	64.42
5	13.0	72.0	72.21
6	14.0	78.0	80.00
7	15.0	85.0	87.79
8	16.0	92.0	95.58
9	17.0	100.0	103.36
10	18.0	108.0	111.15
11	19.0	118.0	118.94
12	20.0	128.0	126.73
13	22.0	142.0	142.30
14	24.0	155.0	157.88
15	26.0	170.0	173.46
16	28.0	185.0	189.03
17	30.0	205.0	204.61
18	32.0	225.0	220.18
19	35.0	250.0	243.55

Gradient Descent - Cost vs Iterations

